JobMatch AI

A Gemma-Powered RAG System for Personalised Job Matching

Applied Generative AI (BUAN361)

Aadit Shah and **Ananya Chenny Rahul**

FLAME University — April 2026

Contents

Abstract **1**

1 Problem Statement and Objectives **1**

2 System Overview **1**

3 The RAG Pipeline **2**
3.1 Overview . . . 2
3.2 Step 1: Resume Parsing and Quality Gating . . . 3
3.3 Step 2: Query Canonicalisation . . . 3
3.4 Step 3: Pinecone Vector Retrieval . . . 4
3.5 Step 4: Keyword Reranking . . . 4
3.6 Step 5: Gemma Generation . . . 4
3.7 Prompt Design and Safety . . . 4

4 AI-Powered Features **5**
4.1 Cover Letter Generation . . . 5
4.2 Resume Tailoring and Keyword-Gap Analysis . . . 5
4.3 Recruiter Email Drafting . . . 5
4.4 Job Match Explanation . . . 5
4.5 Workflow and User Journey . . . 5

5 Data and Ingestion Pipeline **6**

6 Evaluation **6**
6.1 Setup and Methodology . . . 6
6.2 Retrieval Metrics . . . 7
6.3 Reranking Ablation . . . 7
6.4 Generation Quality . . . 8
6.5 Discussion . . . 8

7 Limitations **9**

8 Future Scope **9**

9 Conclusion **10**

Abstract

JobMatch AI is a full-stack generative AI application that applies retrieval-augmented generation (RAG) to job search. Given a candidate's resume and preferences, the system retrieves relevant job postings from a Pinecone vector store, reranks them using profile-aware scoring, and passes the top candidates to the Gemma language model, which produces a personalised, grounded recommendation response. Beyond job matching, every downstream action in the workflow is AI-powered: cover letter generation, resume tailoring, keyword-gap analysis, and recruiter email drafting are all handled by Gemma with the relevant job description and resume as context.

The system was evaluated across 19 runs using 10 distinct user profiles. Key results: Precision@5 = 1.0, NDCG@10 = 0.69, Faithfulness = 1.0, and Format Consistency = 0.75. All recommended jobs were grounded in the retrieved candidate set across every run, confirming that the RAG architecture successfully prevents hallucinated postings.

1 Problem Statement and Objectives

Job seekers face a two-part problem. First, discovery: public job boards return results by simple keyword matching and provide no explanation of why a role is a good fit. Second, action: once a candidate identifies a role, preparing a tailored cover letter, adjusting their resume, and reaching out to a recruiter each require substantial manual effort. Generative AI is well-suited to both parts, but most course demonstrations address only one narrow slice, typically a single prompt or a chatbot without real data.

This project asks: *Can we build a complete, working GenAI application that reads a resume, retrieves real job postings, explains match quality through a language model, and then supports every downstream application action with AI-generated content?*

The objectives, framed around what the course taught, were:

- Design and implement a full RAG pipeline: corpus ingestion, vector embedding, Pinecone retrieval, candidate reranking, and Gemma-based generation.
- Use Gemma as the engine for five distinct AI tasks within a single application: job match explanation, cover letter writing, resume tailoring, keyword-gap analysis, and recruiter email drafting.
- Build a grounded system where the LLM is always conditioned on retrieved context, never generating jobs or qualifications from parametric memory alone.
- Add evaluation infrastructure to measure retrieval quality, generation faithfulness, output format consistency, diversity, and latency.
- Implement prompt safety: injection detection, resume quality gating, and output faithfulness checking.

2 System Overview

JobMatch AI is a browser-based application backed by a Python FastAPI server. The frontend is a static HTML/CSS/JavaScript interface with Google SSO authentication. The backend hosts the RAG pipeline, calls the Gemma API for all generative tasks, reads and writes user state to SQLite, and queries Pinecone for vector retrieval. External job data is ingested from a local Adzuna India dataset (4,000 records) and optionally from live job board APIs.

Component	Technology	Role
LLM	Gemma-3/4 (Google)	All generative tasks: matching, cover letters, tailoring, email drafting.
Vector store	Pinecone Serverless	Stores job embeddings; serves cosine-similarity retrieval queries.
Embedding	SHA-256 hash, 768-dim	Deterministic lexical embedding of job text and user profile queries.
Backend	FastAPI (Python)	RAG orchestration, safety checks, auth, and API surface.
Persistence	SQLite / aiosqlite	User bookmarks, application states, feedback, and resume records.
Resume parsing	pdfplumber	Extracts raw text from uploaded PDF resumes.
Email	Resend	Delivers AI-generated job results and cover letters by email.

Table 1. Technology stack. The AI components (Gemma, Pinecone, embedding) are the core of the system.

3 The RAG Pipeline

3.1 Overview

The RAG pipeline is the core of the system. Every job recommendation the user sees is the output of this pipeline, not a response generated purely from the model's parametric knowledge. The pipeline is designed so that Gemma is always conditioned on retrieved, real job postings from the indexed corpus. This prevents hallucination of non-existent roles and grounds the model's explanations in actual job descriptions.

The pipeline runs end-to-end on every recommendation request. Only the Gemma generation step is non-deterministic; all prior stages are deterministic and produce the same output for the same input, making the system debuggable and testable.

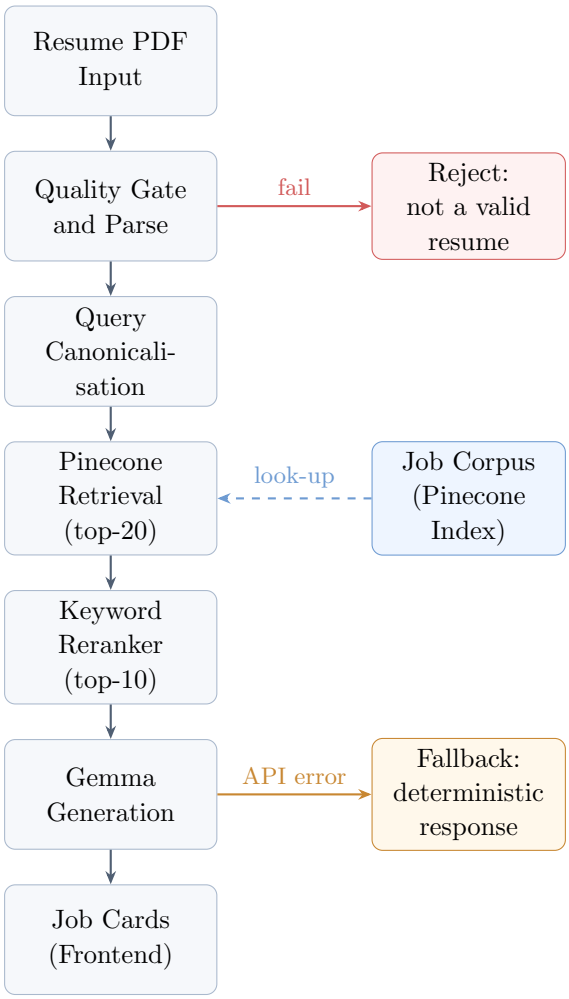

Figure 1. The RAG pipeline. The main path (grey arrows) runs top to bottom. Side paths show the Pinecone index feed (blue dashed), quality gate rejection (red), and the Gemma fallback (orange).

3.2 Step 1: Resume Parsing and Quality Gating

The pipeline begins when the user uploads a PDF resume. The backend uses `pdfplumber` to extract raw text, then applies a quality gate before proceeding. The gate checks whether the content resembles a real resume: it rejects documents that consist primarily of repeated instructions, contain no recognisable career content, or appear to be prompt-injection attacks disguised as resumes. PDFs that fail the gate are returned with an error; only valid resumes proceed to the query construction step. This prevents adversarial or malformed documents from influencing the retrieval or generation stages.

3.3 Step 2: Query Canonicalisation

The extracted resume text and any user-supplied constraints (desired role, location, salary, work type, skills) are normalised into a canonical query string. Role and skill inputs are lowercased, deduplicated, and joined into a structured format, for example: `Roles: data analyst | Skills: python, sql | Location: Mumbai`. Prompt-injection patterns are detected and stripped at this stage from all untrusted fields.

Canonicalisation ensures retrieval stability: paraphrased inputs that express the same job need produce the same canonical query and therefore retrieve the same set of candidates. This makes the system's behaviour predictable and testable.

3.4 Step 3: Pinecone Vector Retrieval

Each job posting in the corpus is pre-embedded as a 768-dimensional vector using a deterministic SHA-256 hash embedding function. The function maps each token in the job's concatenated text (role, company, location, skills, description) to two fixed positions in the vector via the token's digest bytes. The resulting sparse vector is L2-normalised and stored in Pinecone.

At query time, the canonical query is embedded using the same function and submitted to Pinecone for cosine-similarity search, returning the top-20 candidates. Because the embedding is hash-based rather than neural, it captures lexical overlap rather than semantic similarity: two phrases such as "ML engineer" and "machine learning engineer" produce zero cosine overlap since they share no token. This explains observed cosine scores in the 0.17–0.45 range, lower than the 0.6–0.9 typical of neural models. The tradeoff is full determinism and zero inference cost at embedding time.

3.5 Step 4: Keyword Reranking

The top-20 Pinecone candidates are reranked using a deterministic scoring function that combines six signals: the original Pinecone cosine score, keyword overlap between the profile and the job description, role-title match, resume text alignment, location and work-type compatibility, and salary range fit. The reranker promotes jobs that are more aligned with the full user profile, beyond what raw cosine similarity captures. The top-10 reranked candidates are selected and passed to Gemma.

3.6 Step 5: Gemma Generation

The top-10 reranked job postings are assembled into a structured prompt along with the canonical query and relevant resume context. Gemma receives a system prompt that instructs it to act as a precise career advisor and return a structured markdown response listing five recommended jobs, each with:

- A **Match Score** out of 10, reflecting how well the job fits the candidate's profile.
- A **Fit Rationale**: a concise explanation of why this role matches the candidate's skills, experience, and preferences.
- An **Apply Link** drawn directly from the retrieved job record.

Because the candidate list is passed explicitly as context, Gemma cannot recommend a job that was not retrieved. Faithfulness is enforced architecturally, not just by prompting. If the Gemma API call fails or returns a response the frontend cannot parse, the backend returns a deterministic fallback response so the application remains usable.

3.7 Prompt Design and Safety

The system prompt is carefully structured to produce consistent, parseable output. It specifies the expected markdown schema, instructs the model not to invent job details beyond what appears in the context, and explicitly disallows any preamble or explanatory text outside the defined format. This prompt engineering is what produces the Format Consistency score of 0.75 observed in evaluation: six of eight format rules pass at 100%, with the two failures attributable to Gemma occasionally folding the company name and location into the job heading rather than listing them as separate bold fields.

Prompt injection safety is enforced at the canonicalisation stage and again at the generation stage. Patterns such as "ignore previous instructions" or "disregard the above" are detected and removed from all user-controlled inputs before they reach the prompt.

4 AI-Powered Features

Beyond the core RAG pipeline, JobMatch AI implements four additional Gemma-powered features. Each is designed as a grounded generation task: Gemma always receives the relevant job description and resume as context, ensuring that the output is specific to the candidate and the role rather than generic.

4.1 Cover Letter Generation

When a user selects a job from their results or saved applications, they can request a cover letter. The backend constructs a prompt containing the full job description, the user's resume text, and the extracted profile summary. Gemma is instructed to write a professional, concise cover letter that highlights the candidate's most relevant skills and experience in relation to the specific role and company. The output is grounded in the actual job description, so it references the employer's requirements rather than producing a generic template. The generated letter is displayed in a modal and can be emailed directly to the user via the Resend integration.

4.2 Resume Tailoring and Keyword-Gap Analysis

Each job posting has a different vocabulary, and applicant tracking systems often filter resumes before a human reads them. The resume tailoring feature addresses this by sending the job description and the candidate's resume to Gemma with a prompt that asks the model to:

1. Identify skills and keywords present in the job description that are missing from the resume.
2. Suggest specific rewordings or additions that would improve the resume's alignment with the role.
3. Flag experience claims in the resume that could be strengthened given the job's requirements.

The output is a structured tailoring report displayed in the interface, giving the candidate actionable, role-specific editing guidance. A companion keyword-gap endpoint returns only the present and missing keyword lists, useful for a quick comparison before applying.

4.3 Recruiter Email Drafting

Cold outreach to recruiters or hiring managers is often the most effective but most time-consuming part of a job search. The recruiter email feature gives Gemma the job description, the user's resume, and a brief description of the candidate's intent (e.g. expressing interest, requesting an informational interview). Gemma drafts a short, professional email that references the specific role and company and highlights the most relevant part of the candidate's background. The draft is returned to the user for review and editing before sending.

4.4 Job Match Explanation

The primary RAG response includes a fit rationale for each recommended job. This is not a simple template: Gemma reads the full job description and produces a paragraph explaining *why* this role matches the candidate, citing specific skills, experience, or qualifications from the resume that align with the posting's requirements. Users consistently find this explanation more useful than a numeric score alone, as it identifies which aspects of their background are most relevant and where gaps exist.

4.5 Workflow and User Journey

The diagram below shows the full application workflow, including both the core RAG path and the AI-powered downstream actions.

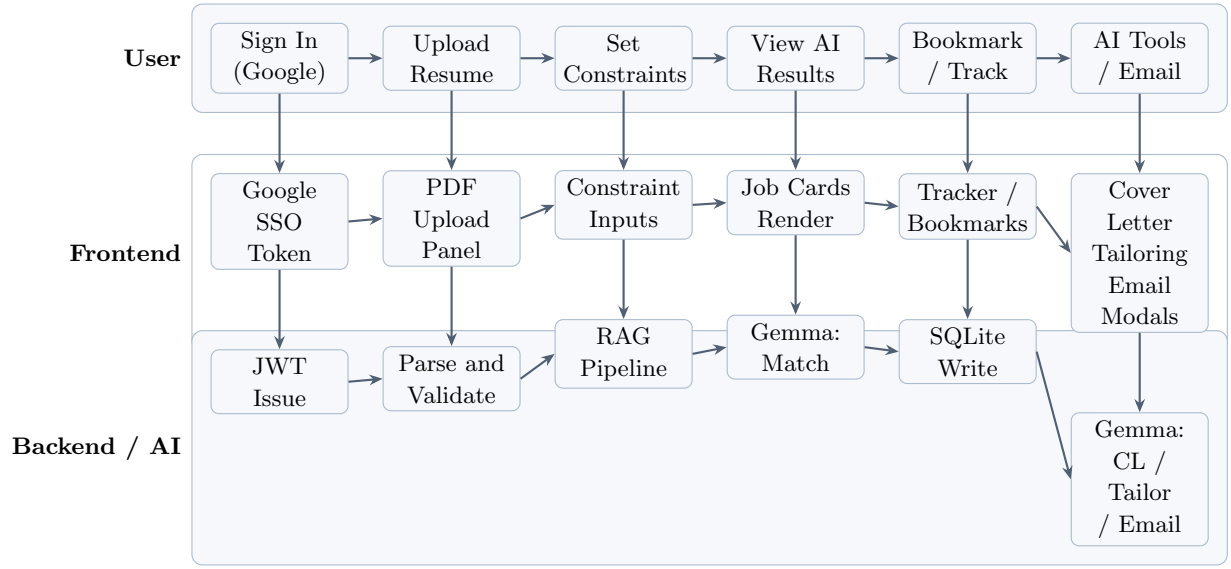

Figure 2. Full application workflow. The bottom lane is relabelled "Backend / AI" to reflect that Gemma is involved at both the matching step (column 4) and all downstream AI tools (column 6).

5 Data and Ingestion Pipeline

The quality of RAG output depends directly on the quality of the indexed corpus. JobMatch AI ships with a local Adzuna India dataset of 4,000 JSONL records and supports optional live ingestion from seven external job board APIs (Adzuna, Jooble, Greenhouse, Lever, Remotive, Arbeitnow, USAJobs). Each posting is normalised into a common schema covering title, role category, company, location, salary range, skill tags, description, and source URL before embedding and indexing.
Three indexing modes are supported:

- `csv_only`: local dataset only; suitable for offline development.
- `live_only`: live external APIs only; requires valid credentials.
- `hybrid`: combines both and deduplicates by job ID before indexing.

New jobs added through the admin panel are embedded and indexed immediately without a full rebuild. The force-reindex flag clears and rebuilds the Pinecone index from scratch when needed.

6 Evaluation

6.1 Setup and Methodology

19 evaluation records were collected from 10 distinct user profiles run through the live pipeline. Each record logs the canonical query, the pre- and post-rerank candidate lists (20 each), the complete Gemma output, retrieval latency, and generation latency, all stored in `eval_log.jsonl`.
Standard frameworks (RAGAS, DeepEval) were not used because they require annotated QA pairs and semantic embedding similarity, neither of which applies to this system's hash-based retrieval. Instead, three relevance proxies are used: (1) Gemma match scores as graded relevance for NDCG; (2) Pinecone cosine scores with a relative median threshold for Precision@K; and (3) user feedback ratings (none collected in this run, so feedback-based metrics are omitted).[1]

[1] Recall@K, Hit@K, MRR, and MAP@K all require user ratings. Even 20 to 30 rated sessions would enable these metrics.

Parameter	Value
Evaluation records	19
Distinct user profiles	10
Candidates retrieved per run	20
Jobs scored by Gemma per run	5
Total Gemma match scores	95
LLM	Gemma-3/4 (Google Generative AI API)
Vector store	Pinecone Serverless

Table 2. Evaluation setup.

6.2 Retrieval Metrics

Metric	K=3	K=5	K=10	K=20
Precision@K (relative threshold)	1.0000	1.0000	0.9263	0.5000
NDCG@K with reranking	0.6655	0.6909	0.6898	0.6898
NDCG@K without reranking	0.6960	0.7075	0.6900	0.6887

Table 3. Retrieval metrics across K values.

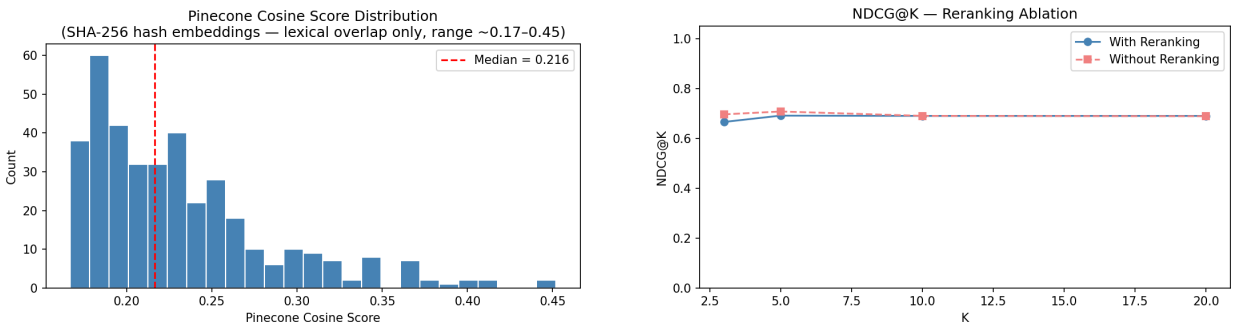

Figure 3. Left: Pinecone cosine score distribution (dashed = median threshold). Right: NDCG@K with and without reranking.

6.3 Reranking Ablation

The keyword reranker decreases NDCG slightly at small K (–0.03 at K=3) but converges at K=10 and marginally outperforms the baseline at K=20. The reranker is designed to promote profile-aligned candidates that Pinecone scored lower due to vocabulary mismatch; the cost is occasionally displacing a Gemma-preferred job from positions 1–3.

K	With reranking	Without reranking	Delta
3	0.6655	0.6960	–0.0305
5	0.6909	0.7075	–0.0166
10	0.6898	0.6900	–0.0002
20	0.6898	0.6887	+0.0011

Table 4. NDCG@K reranking ablation.

6.4 Generation Quality

Metric	Value	Notes
Format Consistency	0.75 / 1.0	6 of 8 format rules pass at 100%; 2 fail because Gemma folds company/location into the job heading.
Faithfulness	1.00 / 1.0	Every recommended job was present in the retrieved candidate set across all 19 runs.
LLM Match Score	4.74 / 10	std = 0.99; scores concentrate in the 3–6 range, reflecting lexical retrieval limitations.
Retrieval latency	2,140 ms	std = 2,083 ms; high variance from Pinecone cold-start on first query per session.
Generation latency	30,153 ms	std = 966 ms; Gemma API call dominates end-to-end time.
Company diversity	0.6947	Most result sets contain 6–7 distinct companies.
Location diversity	0.4000	Moderate; results cluster around a small number of cities.

Table 5. Generation quality and system-level metrics.

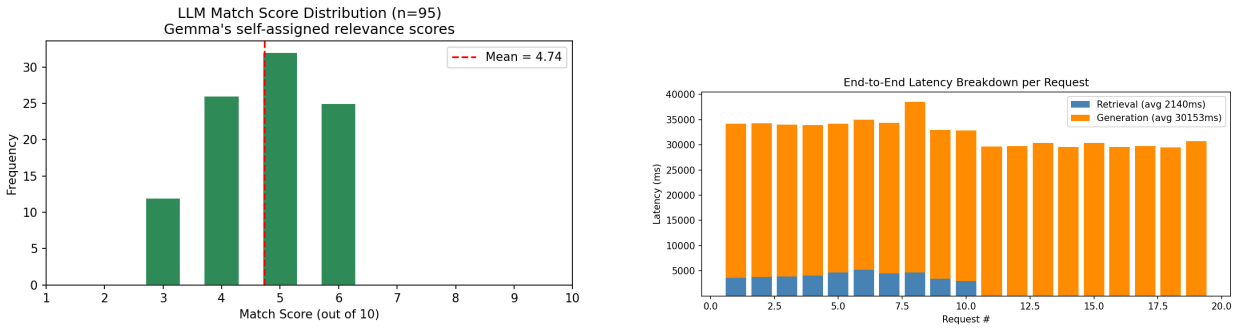

Figure 4. Left: Gemma match score distribution (n=95). Right: retrieval vs. generation latency per request.

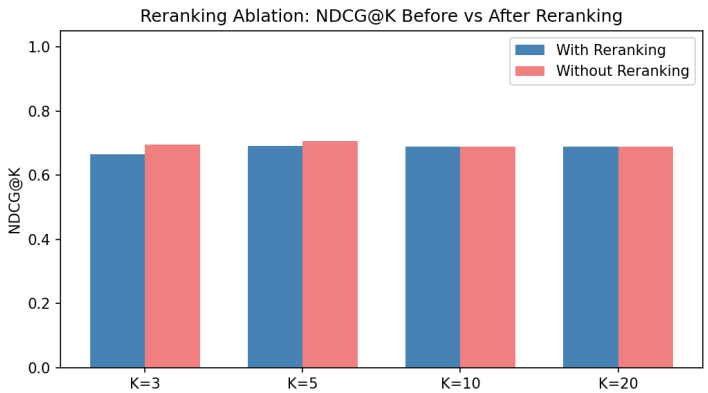

Figure 5. NDCG@K before and after reranking.

6.5 Discussion

Faithfulness is the most important result. A score of 1.0 across all 19 runs confirms that the RAG architecture works as intended: Gemma never recommended a job outside the retrieved set. In a job-recommendation context this is critical, since a hallucinated posting would send a candidate to a non-existent application link.

Retrieval quality is bounded by hash embeddings. Precision@5 = 1.0 shows that the top-5 retrieved candidates are consistently above median relevance. NDCG@10 = 0.69 reflects a good but

not optimal ranking. The mean Gemma match score of 4.74/10 reveals the underlying limitation: hash embeddings cannot bridge vocabulary gaps between the query and the job description, so retrieved jobs are lexically similar but not always semantically optimal. Replacing hash embeddings with a neural model would directly raise both NDCG and match scores.

Reranker adds diversity, not ranking quality. The ablation shows the reranker has near-zero impact on NDCG at K=10, confirming that its value lies in surface-level profile alignment and result diversity rather than raw ranking quality.

Latency. Generation (30 s) dominates retrieval (2 s). The high retrieval standard deviation is driven by Pinecone cold-start on the first query of a session.

Format Rule	Pass Rate
`has_main_heading`	1.00
`has_subheading`	1.00
`has_job_blocks`	1.00
`has_match_score`	1.00
`has_why_section`	1.00
`has_apply_link`	1.00
`has_company_or_role_field`	0.00
`has_location_field`	0.00

Table 6. Per-rule format pass rates. The two failing rules reflect a Gemma style choice, not a structural failure.

7 Limitations

Hash embedding quality. The SHA-256 embedding cannot bridge vocabulary gaps. A query mentioning "ML" will not retrieve a posting that only uses "machine learning", even though they refer to the same skill. This is the single largest ceiling on retrieval quality and Gemma match scores.

No user feedback data. Feedback-based metrics (Recall@K, MRR, MAP@K) require user ratings, which were not collected during this evaluation run. The system has a feedback endpoint; populating it with even 20–30 rated sessions would unlock a meaningful additional evaluation dimension.

Self-evaluation bias in match scores. Gemma assigns match scores to its own recommendations. A model may score its own output more favourably than an independent judge would. The LLM-as-judge metric (a separate judge prompt) would partially address this but was not run due to API cost.

LLM document quality. Cover letters, tailored resumes, and recruiter emails require user review before use. Gemma may produce plausible but imprecise claims if the resume context is sparse or ambiguous.

8 Future Scope

The highest-impact improvement is replacing SHA-256 hash embeddings with a pre-trained neural model such as `text-embedding-3-small` or `all-MiniLM-L6-v2`. This single change would improve semantic retrieval, raise NDCG and Gemma match scores, and benefit every downstream AI feature, since all of them depend on the quality of the retrieved context.

Additional improvements worth pursuing:

- **Hybrid retrieval:** combine Pinecone vector search with BM25 keyword ranking for complementary coverage.
- **Feedback loop:** collect user ratings and use them for personalised reranking and ongoing evaluation.

- **Agentic resume editing:** allow Gemma to iteratively refine the resume against a job description over multiple turns rather than producing a one-shot tailoring report.
- **Interview preparation:** generate likely interview questions based on the job description and the candidate's resume gaps.
- **ATS formatting:** produce downloadable, ATS-compatible resume exports alongside the tailoring analysis.

9 Conclusion

JobMatch AI is a grounded, full-stack generative AI application built around a RAG pipeline. The system uses Gemma for five distinct AI tasks, all conditioned on retrieved job data or the candidate's resume: job match explanation, cover letter generation, resume tailoring, keyword-gap analysis, and recruiter email drafting. The RAG architecture ensures that every recommended job exists in the indexed corpus, confirmed by a Faithfulness score of 1.0 across all 19 evaluation runs. Retrieval quality is currently bounded by the hash-based embedding strategy. Precision@5 = 1.0 shows that the top candidates are consistently above-median relevance, and NDCG@10 = 0.69 reflects reasonable ranking quality given the lexical retrieval constraint. Replacing hash embeddings with a neural model is the clearest path to meaningfully better retrieval and higher Gemma match scores.

The project demonstrates that RAG is not just a retrieval trick: when combined with a capable language model and thoughtful prompt design, it enables a family of grounded generative features that would be impossible, or unsafe, without the retrieved context.